Multi-client Chatroom with LLM Integration - Assignment

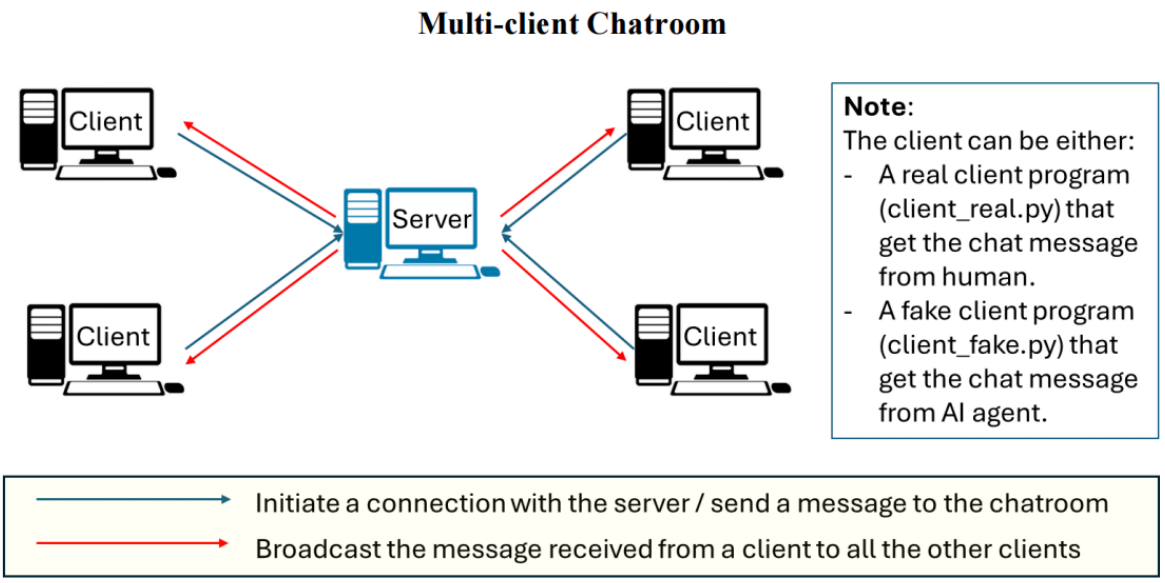
Overview

This assignment requires implementing an online chatroom supporting multiple clients in two phases based on Python socket programming and multi-threading technology. The core objectives are to master TCP network connection management, multi-threaded concurrent processing, integrate a Large Language Model (LLM) with the chatroom, and finally compare the conversational characteristics of humans and LLMs through video demonstrations and in-class guessing sessions. You are required to develop a server, human clients, and integrate an LLM client in the second phase, recording demonstration videos throughout for classmates to identify human/LLM identities in class.

Objectives

Upon completion of this assignment, you will be able to:

1. Use Python Socket programming to establish and manage TCP connections for real-time network communication;
2. Master multi-threading technology to handle concurrent tasks in network applications (multi-client connections, message sending/receiving, input monitoring, etc.);
3. Integrate a Large Language Model (LLM) with a network chatroom to simulate the behavior of an AI conversationalist;
4. Analyze the conversational characteristics of humans and LLMs, and design AI conversation logic that is closer to human behavior;
5. Possess the ability to debug, test, and demonstrate network programs.



Submission Content and Deadline

UPDATE : Please submit your documents according to the following file organization and naming rules.

Group assignment, with a recommended group size of 2-3 people. **Only one group member needs to submit all assignment files uniformly to Canvas** (Link: https://canvas.duke.edu/courses/76875/assignments/355195) .

Submit ONE single ZIP file only, named in format:

```
name1_name2_pa2.zip
```

Inside the ZIP file, structure strictly as below:

```
name1_name2_pa2.zip
├─ slides/                      # Presentation slides
├─ src/                         # Source code folder
│  ├─ server.py                 # Server program
│  ├─ client_human.py           # Human client program
│  └─ client_llm.py             # LLM client program
└─ video/                       # All videos
   ├─ stage1_2human.mp4         # stage1 video - 2 human
   ├─ stage1_3human.mp4         # stage1 video - 3 human
   ├─ stage2_2human_1llm.mp4    # stage2 video - 2 human & 1 llm
   ├─ stage2_3llm.mp4           # stage2 video - 3 llm
   └─ guess_video.txt           # The video introduction for guess game
```

ALL DEADLINE: **May 3, 23:59**

After voting, the presentation session in class will be canceled and replaced with **Recorded video presentation**.

Video naming rules

A total of **four videos** shall be uploaded, with **2 videos for Stage 1** and **2 videos for Stage 2** respectively.

For the guessing game task, please specify the two selected videos in `guess_video.txt` with the following information:

1. Original video name
2. Real identity of each conversation participant
3. Number of human users and LLM participants in each video

For all other videos, follow the naming convention below:

```
stage{t}_{x}human{y}llm.mp4
```

- `t`: the stage number of the video
- `x`: the number of human participants in the conversation
- `y`: the number of LLM participants in the conversation

Stage 1: Human-Only Multi-Client Chatroom

Core Requirements

Implement a chatroom that supports simultaneous connections from multiple human clients. The server is responsible for managing connections and broadcasting messages. Clients support human users to input, receive, and display messages from other users, enabling **real-time conversations between humans only**.

Programs to be Developed

In this phase, you need to complete 2 Python programs, which are required to run independently without syntax errors, and handle common exceptions (such as connection interruptions, empty messages, abnormal client exit, etc.).

1. Server Program: `server.py`

Core Functions:

- Listen for incoming TCP connections on a specified IP and port;
- Maintain a list of connected clients, using lock mechanisms to prevent list exceptions caused by multi-thread competition;
- Create an independent thread for each client to specifically handle message reception from that client;
- Broadcast client status messages: new user joining, user exiting;
- Receive chat messages sent by clients and broadcast them to all other connected clients;
- Handle abnormal disconnection of clients, promptly remove them from the client list, and broadcast exit messages.

Development Tips:

- Import the `socket` and `threading` modules, and create a socket based on the TCP protocol (SOCK_STREAM);
- Use `threading.Lock()` to achieve thread-safe access to the client list;
- Capture exceptions for socket operations and client communication (such as `ConnectionResetError`, `OSError`, etc.);
- Set a fixed server IP (e.g., local loopback `127.0.0.1`) and port (e.g., `8888`, avoid using system default ports).

2. Human Client Program: `client_human.py`

Core Functions:

- Initiate a TCP connection to the server, join the chatroom after entering a username;
- Support human users to input chat messages from the console and send them to the server;
- Receive messages broadcast by the server in real time (chat content from other users, notifications of users joining/exiting) and display them in a formatted way;
- Implement **concurrent message sending and receiving**: one thread listens for server messages, and one thread listens for user input to avoid input/display blocking;
- Handle exceptions such as disconnection and server offline, and give friendly prompts.

Development Tips:

- Use `sys.stdout.write` and `sys.stdout.flush` to achieve real-time message display and avoid buffer delays;
- Multi-thread division of labor: the main thread handles user input, and the sub-thread cyclically receives server messages;
- Filter empty messages and prohibit sending empty content to the server;

- The chat display format should be clear, example:

```
Please enter your username: Tom
Welcome Tom to the chatroom!
>> (Waiting for user input)
Jerry: Hello there!
Lucy has joined the chatroom!
>> Hello everyone!
Tom: Hello everyone!
Lucy: Welcome new friend~
Jerry has left the chatroom!
```

Testing and Video Recording

Test Scenarios

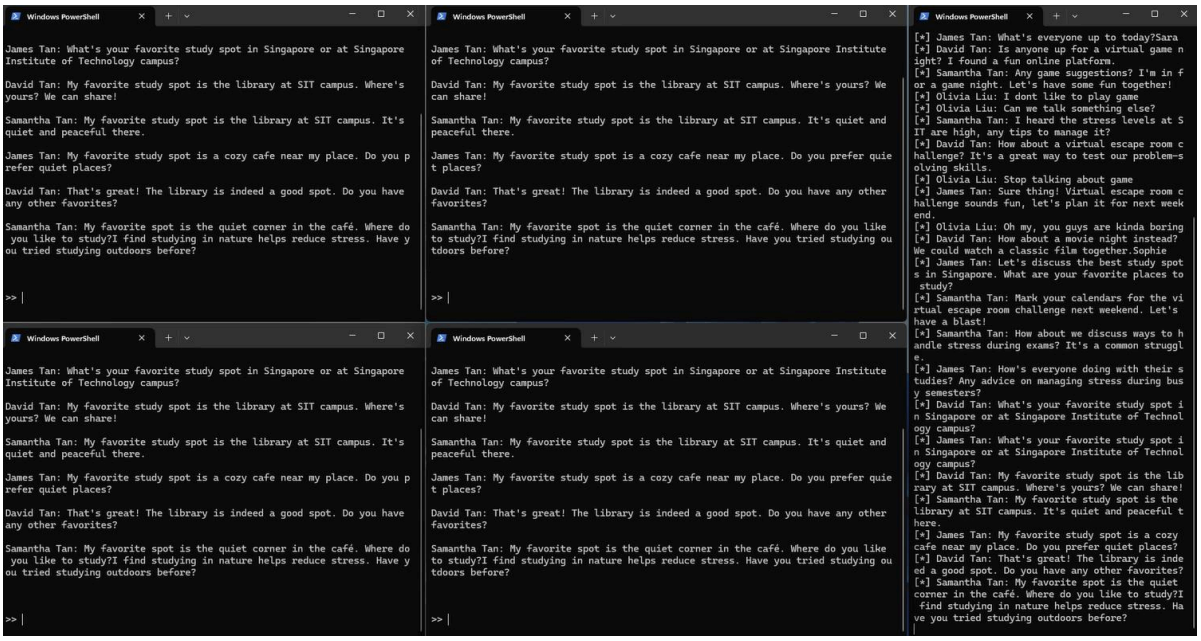
Complete testing of **at least 2 human-only conversation scenarios** to ensure the normal functioning of multi-client concurrent connections, message broadcasting, and status notification functions:

1. Conversation between 2 human clients;
2. Conversation between 3 human clients (including cases of joining/exiting midway).

Video Requirements

1. Record a demonstration video of **within 1 minute** for each scenario, with clear display in the video: interfaces of all clients, usernames, real-time sending/receiving of messages, and notifications of users joining/exiting;
2. Arrange all client windows on the same screen during recording for easy viewing;
3. The video should be free of stutters and black screens, and sound is optional (focus on displaying text interaction).

Please arrange your clients in front of the screen for recording as follows:



Stage 2: Multi-Client Chatroom with LLM Integration

Core Requirements

Based on the `server.py` from Phase 1 (minor adaptations are allowed, no need for reconstruction), develop an **LLM client program** to enable the LLM to join the chatroom as a "virtual user". Support two modes: **human-LLM conversation** and **LLM-LLM conversation**. Complete multi-scenario testing and video recording to prepare for the guessing session.

Programs to be Developed

In this phase, you need to complete 1 new program, namely the LLM client program `client_llm.py`. The server `server.py` can be slightly adapted according to requirements (e.g., compatible with the message format of the LLM client), and the human client `client_human.py` remains unchanged.

Core Functions:

- Follow the same communication protocol as `client_human.py`, initiate a connection to the server, and join the chatroom with a custom AI username;
- Receive all messages broadcast by the server (chat content from human users, status notifications of other users/AIs);
- Generate contextually appropriate replies by calling the LLM interface based on the received chatroom messages, and automatically send them to the server;
- Control the **frequency and delay** of AI replies (simulate human typing and thinking, avoid instant replies to enhance confusion);
- Efficiently manage LLM API calls and capture exceptions (such as API call failures, network timeouts, token exhaustion, etc.);
- Optional: Use a message queue to manage received messages to avoid excessive memory usage.

Development Tips:

- Any LLM can be selected: OpenAI GPT series (refer to official API documentation), domestic large models (such as ERNIE Bot, Tongyi Qianwen);
- Perform simple processing on LLM replies: limit reply length (simulate short human conversations), avoid overly official expressions, and add a slight colloquial tone;
- Set AI reply delay using `time.sleep()` to avoid replying too quickly and exposing the AI identity;
- Capture API call-related exceptions (such as `requests.exceptions.RequestException`), give prompts when calls fail, without affecting client connections;
- Keep the message format consistent with the human client to ensure the server can broadcast normally.

Testing and Video Recording

Test Scenarios

Complete testing of **conversation scenarios integrated with LLM**, considering combinations of single/multiple humans and single/multiple LLMs, and design diverse scenarios to increase the difficulty of guessing:

Examples:

1. Conversation between 2 humans + 1 LLM;
2. Conversation between 3 LLMs (AI-only conversation).

Video Requirements

1. Record a demonstration video of **within 1 minute** for each scenario, with the same recording requirements as Phase 1 (all clients displayed on the same screen, message interaction clearly visible);
2. **Key Requirement**: The display interface of the LLM client is exactly the same as that of the human client, with no identifiers to distinguish AI/human, enhancing the confusion of guessing;
3. The video should include continuous interaction of messages, avoiding meaningless single message sending.

Guess videos

UPDATE :

Keep within 1 minutes per video

1. Each group **selects two** of the submitted videos to ~~play in class~~ upload to canvas (according to the submission requirements) , hiding client type information;
2. You needspecify the two selected videos in `guess_video.txt` with the following information:
    1. Original video name
    2. Real identity of each conversation participant
    3. Number of human users and LLM participants in each video
3. Other classmates act as judges, watch the videos and judge the identity (human/LLM) of each client;

4. Group scoring criteria: The **error rate** of judges' judgments on the group's videos. The higher the error rate (i.e., the AI is closer to humans and difficult to identify), the higher the group's score;
5. Extra points will be awarded for the guessing session.

Recorded video presentation

UPDATE : After voting, the presentation session in class will be canceled and replaced with Recorded video presentation.

Keep within 5 minutes

The recorded video Presentation focus on the multi-client chat room assignment introduction

- Demonstration duration : within 5 minutes for **assignment introduction**, strictly control the time, points will be deducted for overtime as appropriate;
- Presenters : Each group recommends 1-2 members to be responsible for explanation and interaction, with clear division of labor in advance.
- Content :

1. Briefly explain the overall architecture (server + client) and the design and implementation of core functions;
2. Explain the selected LLM model, API calling method, and AI reply optimization design (such as reply delay, colloquial processing, length limit, etc.), and explain how to make AI conversations closer to human behavior;

Development Tools and Reference Materials

1. **Development Environment**: Python 3.8 and above (3.9/3.10 recommended), no need to develop a GUI, only console programs are required;
2. **Core Modules**: socket (network communication), threading (multi-threading), sys (console output), requests (LLM API calling), time (delay control);
3. **LLM API References**: OpenAI official documentation (https://platform.openai.com/docs/quickstart), official open platform documentation of domestic large models;

Notes

1. **Code Originality**: Collaboration within the group is encouraged, and reference to official documentation and technical blogs is allowed, but copying code from other groups is strictly prohibited;
2. **LLM API Usage**: When using third-party LLM APIs, manage API keys by yourself to avoid key leakage. It is recommended to read the key from a configuration file in the code instead of hardcoding;
3. **Port Conflicts**: If you encounter the `Address already in use` error during development, you can change the server port, or use `ss -tulnp` (Linux)/ `netstat -ano` (Windows) to check port occupancy and release it;
4. **Multi-Thread Debugging**: Multi-threaded programs are prone to deadlocks and data race issues. Use lock mechanisms reasonably, and gradually increase the number of clients during debugging to avoid difficulty in locating problems caused by testing multiple clients at once;

5. **Video Recording**: It is recommended to use the system's built-in screen recording tools (such as Windows built-in screen recording, Mac QuickTime) to ensure clear video resolution and unobstructed windows.

Rubrics

- Incomplete implementation of all functions: **< 90**
- Full completion of basic functions: **~92**
- Full completion of extended functions: **> 92**
- Guessing game bonus: **0–2 points**